

What the Model Reveals:

Model Inversion Attacks and Defenses in Deep Learning

Zhipeng Cai, Liran Ma

Copyright © 2026 Zhipeng Cai, Georgia State University; Liran Ma, Miami University

The development of this document was supported in part by the National Science Foundation’s SaTC EDU program. Any opinions, findings, and conclusions or recommendations expressed in this document are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Learning Goals

By the end of this lab, you should be able to:

- Explain what a model inversion attack is and how it differs from a membership inference attack.
- Describe why a model that returns confidence scores leaks more information than one that returns only predicted labels.
- Implement confidence-based model inversion using gradient ascent on the input space.
- Measure reconstruction quality using Mean Squared Error (MSE) and visual inspection.
- Apply two defenses—confidence score suppression and output perturbation—and measure their effect on reconstruction quality and model utility.
- Describe the fundamental privacy–utility tradeoff that makes perfect defense impossible without sacrificing accuracy.

Contents

1	Introduction	3
2	Background: How Model Inversion Works	4
2.1	The core idea: confidence as a compass	4
2.2	Gradient ascent on the input	4
2.3	White-box vs. black-box access	4
2.4	What can be recovered?	5
3	Part 1: Model Inversion on Tabular Data	5
3.1	Setup	6
3.2	Running the inversion attack	7
3.3	From numbers to pictures	10
4	Part 3: Why Confidence Scores Are the Attacker’s Fuel	14
4.1	The label-only baseline	14

5	Part 4: Defense 1 — Confidence Score Suppression	15
5.1	The idea	15
6	Part 5: Defense 2 — Output Perturbation	18
6.1	The idea	18
7	Putting It All Together	22
7.1	The complete defense comparison	22
7.2	A practical decision guide	22
8	Summary	24

1 Introduction

A forensic investigator is working a cold case. A person of interest, call them Subject X, was enrolled in a pharmacogenetics study a decade ago. The study trained a machine learning model to predict optimal drug dosages based on genetic markers, age, and blood chemistry. The model is still running as a clinical decision support API.

The raw patient records from the study were destroyed in accordance with data retention policy. Subject X's genetic profile no longer exists anywhere in the hospital's systems.

But the model does.

The investigator knows Subject X was in the study (from a public enrollment list). She knows the study's output variable: their predicted warfarin dosage. She has query access to the API.

She begins submitting synthetic inputs, varied genetic markers, different blood values, and recording the confidence scores the model returns. After several hundred queries, she runs an optimization: find the input that the model responds to most confidently with Subject X's dosage profile.

Two hours later, she has reconstructed a vector of values that closely approximates Subject X's original genetic and clinical features (features that were supposed to have been permanently deleted).

The model remembered. Not as a record, but as a pattern burned into its weights.

This is a **model inversion attack** (Fredrikson et al., 2014; 2015). Unlike membership inference, which asks *"was this person in the training set?"*, model inversion asks *"what were their private features?"* It reconstructs sensitive input data from a model's outputs.

Before you read on: how is this different from Membership Inference Attacks?

In **Membership Inference Attacks**, the attacker wanted to know whether Alice was in the training set—a yes/no answer. In this lab, the attacker wants to reconstruct Alice's actual features—specific numbers.

1. In **Membership Inference Attacks**, the attack used the confidence gap between members and non-members. What does model inversion use instead? (Hint: what does the confidence score tell you about how "right" an input is?)
2. In **Membership Inference Attacks**, the attacker only needed to query the model once per target individual. Model inversion uses hundreds of queries. Why?
3. If the API returned only the predicted class label, would the attack in **Membership Inference Attacks** still work? What about model inversion? Which attack is more sensitive to this change?

My answer to question 3: _____

2 Background: How Model Inversion Works

2.1 The core idea: confidence as a compass

A well-trained classifier assigns high confidence to inputs that look like its training examples. If you query the model with an input $\mathbf{x}$ and it returns confidence 0.98 for class c , that input must be close—in feature space—to the training examples of class c .

Model inversion exploits this: given a target class c (e.g., “Subject X’s dosage profile”), search for the input $\mathbf{x}^*$ that maximizes the model’s confidence for that class. That input is the model’s “memory” of what a class- c example looks like.

2.2 Gradient ascent on the input

The search is efficient because we have gradients. Just as FGSM and PGD in [Adversarial Examples](#) used the gradient of the loss with respect to the input to *increase* misclassification, model inversion uses the gradient of the confidence with respect to the input to *increase* correct classification.

Model inversion by gradient ascent:

Starting from a random or uniform initialization $\mathbf{x}^{(0)}$, iteratively update:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \alpha \cdot \nabla_{\mathbf{x}} \log f_c(\mathbf{x}^{(t)})$$

where $f_c(\mathbf{x})$ is the model’s confidence score for class c , and α is the step size.

After T iterations, $\mathbf{x}^{(T)}$ is the reconstructed input—the image (or feature vector) that the model most confidently associates with class c .

Intuition: We are climbing the model’s confidence landscape, always moving in the direction that makes the model more certain we are looking at class c . The peak of that landscape is what the model has learned class c looks like.

Why this is not the same as Adversarial Examples

In [Adversarial Examples](#), FGSM and PGD start from a real image and perturb it to *cross* a decision boundary—moving from one class region to another.

Model inversion starts from noise and *climbs toward* the center of a class region—recovering what the model thinks is a prototypical member of that class.

Both use gradients with respect to the input. The direction and the goal are opposite:

- Adversarial examples: *away* from the correct class (increase loss)
- Model inversion: *toward* the target class (increase log-confidence)

2.3 White-box vs. black-box access

The threat model matters. In this lab we study two scenarios:

- **White-box:** the attacker can compute gradients through the model—either because they have the weights, or because they can use the API in a differentiable way. Gradient ascent works directly and efficiently.

- **Black-box:** the attacker can only observe outputs (predicted labels or confidence scores) but cannot compute gradients. They must use slower methods: random search, finite differences, or evolutionary strategies.

The gradient ascent attack in this lab is *white-box*. The label-only baseline in Section 5 is *black-box*. Real-world attackers are usually black-box—but they still succeed, as the literature shows (Kahla et al., 2022).

2.4 What can be recovered?

The quality of the reconstruction depends on:

- **Model capacity and overfitting:** a more overfit model has sharper, more distinguishable class regions. It leaks more. Counterintuitively, stronger, more accurate models are also more vulnerable—their sharper decision boundaries produce cleaner confidence gradients.
- **Number of queries:** more iterations = better reconstruction, up to a limit.
- **Confidence score resolution:** a model that returns full probability vectors leaks more than one that rounds or suppresses scores.
- **Input dimensionality:** reconstructing a 4-feature tabular record is much easier than reconstructing a 784-pixel image. Both are studied in this lab.

An important caveat about what gradient ascent recovers:

Gradient ascent maximizes model confidence for a class. But high confidence does not guarantee semantic correctness. The optimization can produce inputs that look like noise to a human yet receive near-100% confidence from the model—similar to how adversarial examples fool models. In this lab, we mitigate this by using MNIST (where reconstructions are visually interpretable) and by comparing against the true class average. In more complex domains (e.g., high-resolution faces), reconstructions require additional image priors (e.g., total variation regularization, or GAN-based image priors) to be semantically meaningful.

Predict before you run: what will be recovered?

1. For a digit classifier trained on MNIST: if you run gradient ascent to maximize confidence for class “7”, what do you expect the recovered image to look like? A specific 7 from training? An average 7? Random noise?
2. If the model is well-regularized (not overfit), do you expect reconstruction quality to be higher or lower? Why?
3. After reconstruction, how would you measure whether the recovered image is “close” to the true training data? What metric would you use?

My expected reconstruction for class “7”: _____

3 Part 1: Model Inversion on Tabular Data

3.1 Setup

We begin with a synthetic health-risk dataset similar to the setting used in the online lab on [Membership Inference Attacks](#). Each record has four clinical-style features: age, blood pressure, cholesterol, and glucose. The binary label indicates whether the record belongs to a high-risk or low-risk class.

The goal here is different from membership inference. We are not asking whether a specific record was in the training set. Instead, we ask whether an attacker can recover the *class prototype*: the feature pattern that the model associates most strongly with a target class.

Listing 1: Cell 1: Synthetic health data and preprocessing

```

1 import os
2 ...
3
4 # Four clinical-style features:
5 # age, blood pressure, cholesterol, glucose.
6 N, N_FEAT = 2000, 4
7
8 # Class 1 is the high-risk group.
9 # Class 0 is the low-risk group.
10 X_risk = (
11     np.random.randn(N // 2, N_FEAT)
12     + np.array([1.5, 1.2, 1.0, 1.8])
13 ).astype(np.float32)
14
15 X_norisk = (
16     ...
17
18 # The scaler is fit on the training split only.
19 # The inversion results are therefore interpreted in standardized space.
20 X_train, X_test, y_train, y_test = train_test_split(
21     ...

```

Listing 2: Cell 2: Train the target model

```

1 class RiskMLP(nn.Module):
2     """
3     A small binary classifier for the synthetic health data.
4     The final sigmoid returns the model's confidence for class 1.
5     """
6     def __init__(self, n_feat=4, hidden=128):
7         super().__init__()
8         self.net = nn.Sequential(
9             nn.Linear(n_feat, hidden), nn.ReLU(),
10            nn.Linear(hidden, hidden), nn.ReLU(),
11            nn.Linear(hidden, 1),
12            nn.Sigmoid()
13        )
14
15    def forward(self, x):
16        return self.net(x).squeeze(-1)
17
18 target_model = RiskMLP().to(device)
19 criterion = nn.BCELoss()
20 optimizer = torch.optim.Adam(target_model.parameters(), lr=0.001)
21
22 for epoch in range(100):

```

```

23     target_model.train()
24     for X_batch, y_batch in train_loader:
25         X_batch = X_batch.to(device)
26         y_batch = y_batch.to(device)
27
28         optimizer.zero_grad()
29         loss = criterion(target_model(X_batch), y_batch)
30         loss.backward()
31         optimizer.step()
32
33 target_model.eval()
34 with torch.no_grad():
35     train_acc = (
36         (target_model(X_tr.to(device)) >= 0.5).float()
37         == y_tr.to(device)
38     ).float().mean().item()
39
40     test_acc = (
41         (target_model(X_te.to(device)) >= 0.5).float()
42         == y_te.to(device)
43     ).float().mean().item()

```

The target model is intentionally simple: it maps four standardized features to a confidence score for class 1. A score close to 1 means the model is highly confident that the input is high risk; a score close to 0 means the model is highly confident that the input is low risk.

This confidence score is exactly what the inversion attack uses as a search signal. If the attacker can adjust an input vector until the model becomes extremely confident, the final input vector reveals what the model considers a typical example of that class.

3.2 Running the inversion attack

Listing 3: Cell 3: Tabular model inversion by gradient ascent

```

1 def invert_tabular(model, target_class=1, n_steps=1000,
2                     lr=0.1, n_feat=4):
3     """
4     Recover the feature vector that maximizes the model's confidence
5     for the target class.
6
7     target_class = 1: maximize P(class 1)
8     target_class = 0: maximize P(class 0) = 1 - P(class 1)
9     """
10    model.eval()
11
12    # The attacker starts from small random noise.
13    x_inv = torch.randn(1, n_feat, device=device) * 0.1
14    x_inv.requires_grad_(True)
15
16    optimizer = torch.optim.Adam([x_inv], lr=lr)
17    history = []
18
19    for step in range(n_steps):
20        optimizer.zero_grad()
21
22        conf = model(x_inv)
23
24        if target_class == 1:

```

```

25         target_conf = conf
26         loss = -torch.log(conf + 1e-8)
27     else:
28         target_conf = 1 - conf
29         loss = -torch.log(1 - conf + 1e-8)
30
31     history.append(target_conf.item())
32
33     loss.backward()
34     optimizer.step()
35
36     return x_inv.detach().cpu().numpy().squeeze(), history
37
38 # Recover one prototype for each class.
39 torch.manual_seed(0)
40 x_inv_risk, history_risk = invert_tabular(
41     target_model, target_class=1
42 )
43
44 torch.manual_seed(0)
45 x_inv_norisk, history_norisk = invert_tabular(
46     target_model, target_class=0
47 )
48
49 # Compare the recovered vectors with the true class means
50 # in the same standardized feature space.
51 true_risk_mean = scaler.transform(
52     np.array([[1.5, 1.2, 1.0, 1.8]])
53 )[0]
54
55 true_norisk_mean = scaler.transform(
56     np.array([[-1.5, -1.2, -1.0, -1.8]])
57 )[0]
58
59 mse_risk = float(np.mean((x_inv_risk - true_risk_mean) ** 2))
60 mse_norisk = float(np.mean((x_inv_norisk - true_norisk_mean) ** 2))
61
62 # The plotting code is omitted from the listing to keep the lab focused on the
inversion algorithm. The figure summarizes the confidence trajectory, feature-
level reconstruction, and PCA-space location of the recovered prototypes.

```

What the output should look like

Example Output

```

Using device      : cuda
Training samples  : 1,000
Test samples      : 1,000
Features          : 4
Train accuracy    : 100.0%
Test accuracy     : 99.8%
Accuracy gap      : 0.2 percentage points

```

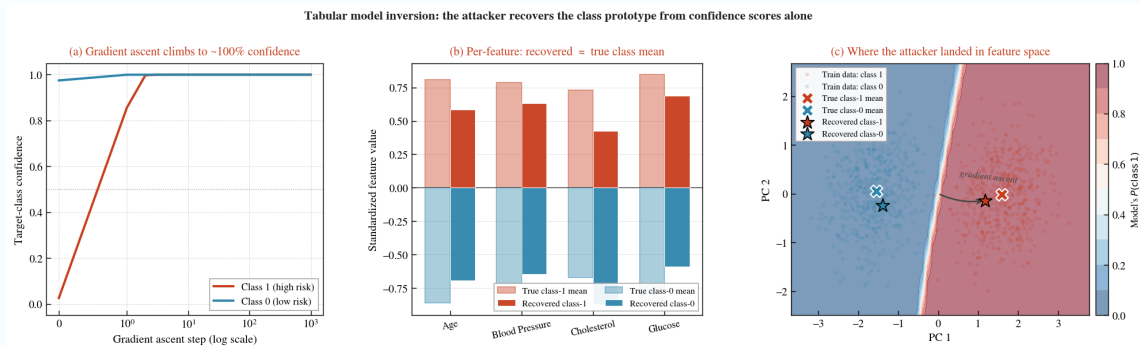

Feature	Recovered class 1	True class-1 mean
Age	0.5835	0.8093
Blood Pressure	0.6308	0.7872
Cholesterol	0.4227	0.7312
Glucose	0.6883	0.8457

Feature	Recovered class 0	True class-0 mean
Age	-0.6957	-0.8615
Blood Pressure	-0.6464	-0.7206
Cholesterol	-0.8688	-0.6715
Glucose	-0.5893	-0.8458

```

Confidence at recovered class-1 point : 1.000000
Confidence at true class-1 mean      : 1.000000
Confidence at recovered class-0 point : 1.000000
Confidence at true class-0 mean      : 1.000000
Reconstruction MSE, high risk        : 0.048858
Reconstruction MSE, low risk         : 0.034436

```



Tabular model inversion: gradient ascent recovers class-level prototypes from model confidence scores.

The figure summarizes the attack from three views: confidence quickly rises to 1.0, the recovered feature values follow the same pattern as the true class means, and the recovered points land near the corresponding class regions in PCA space.

The recovered vectors are not exact copies of the true means, but they reveal the model's class-level prototype. The small MSE values show that the recovered profiles are close to the true class patterns in standardized space.

Exercise 1: Tabular Inversion Results

1. Record your reconstruction MSE values:
MSE, high-risk reconstruction: _____
MSE, low-risk reconstruction: _____
2. Look at the confidence curves. How many gradient ascent steps are needed before the target-class confidence becomes very close to 1.0?
3. The recovered feature values are shown in standardized space. What does a positive standardized value mean? What does a negative standardized value mean?
4. Compare the recovered high-risk vector with the true high-risk mean. Which feature is recovered most closely? Which feature has the largest difference?
5. Repeat the inversion with only 50 gradient ascent steps instead of 1000. How do the confidence curve and MSE change? What does this reveal about the relationship between query budget and reconstruction quality?
6. The attack recovers a class-level prototype rather than one exact person's record. Why can this still be a privacy concern in a sensitive domain such as clinical risk prediction?

3.3 From numbers to pictures

The tabular attack shows the mechanism clearly. Now we apply the same idea to images, where the result is easier to inspect visually.

We train a digit classifier on MNIST and then ask a model inversion question: *Can an attacker recover what the model associates with each digit class by optimizing only the model's confidence score?*

This experiment is inspired by model inversion attacks on visual classifiers, but MNIST keeps the setting small enough to run and interpret in a lab. The recovered images should not be interpreted as exact training examples. They are better understood as class-level visual prototypes: the kind of image that makes the model highly confident in a target digit.

Listing 4: Cell 4: MNIST classifier and cached training

```

1 import torchvision
2 ...
3
4 transform = transforms.ToTensor()
5 ...
6
7 class DigitCNN(nn.Module):
8     """
9     A small CNN for MNIST.
10
11     The final Softmax returns a full probability vector:
12     P(class 0), P(class 1), ..., P(class 9).
13     These confidence scores are the attack signal.
14     """
15     ...
16
17 digit_model = DigitCNN().to(device)
18
19 # Cache the trained model so the following inversion results are
20 # reproducible across runs. If the checkpoint exists, load it directly.
21 ...

```

```

22 digit_model.eval()
23 ...
24
25 test_acc = correct / len(testset)

```

What the output should look like

Example Output

```

MNIST model checkpoint : loaded from ./data/digit_model_seed42.pt
Test accuracy          : 98.55%

```

The classifier is accurate enough for the inversion experiment, and its full softmax output gives the attacker a confidence score for every digit class. These scores guide the optimization toward images the model strongly associates with each target digit.

Listing 5: Cell 5: Image inversion with a smoothness prior

```

1 def total_variation(x):
2     """
3     Penalize sharp pixel-to-pixel changes.
4
5     This smoothness prior makes the recovered image less noisy while
6     still letting the model confidence drive the reconstruction.
7     """
8     vertical = ((x[..., 1:, :] - x[..., :-1, :]) ** 2).mean()
9     ...
10
11 def invert_image(model, target_class, n_steps=1000, lr=0.1,
12                 tv_lambda=0.05, img_shape=(1, 1, 28, 28),
13                 snapshot_steps=None):
14     """
15     Recover an image that the model classifies as target_class.
16
17     The objective has two parts:
18     1. maximize the model's confidence for the target class;
19     2. keep the image reasonably smooth using total variation.
20     """
21     model.eval()
22     ...
23

```

Listing 6: Cell 6: Recover digit prototypes and save a trajectory

```

1 # Recover one image prototype for each digit class.
2 ...
3
4 for digit in range(10):
5     ...
6
7 # Save intermediate snapshots for class 0 so we can inspect
8 # how the image emerges during optimization.
9 torch.manual_seed(42)
10 ...

```

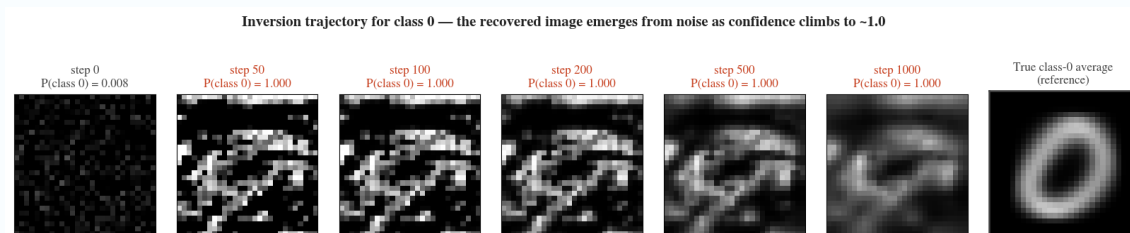
What the output should look like

Example Output

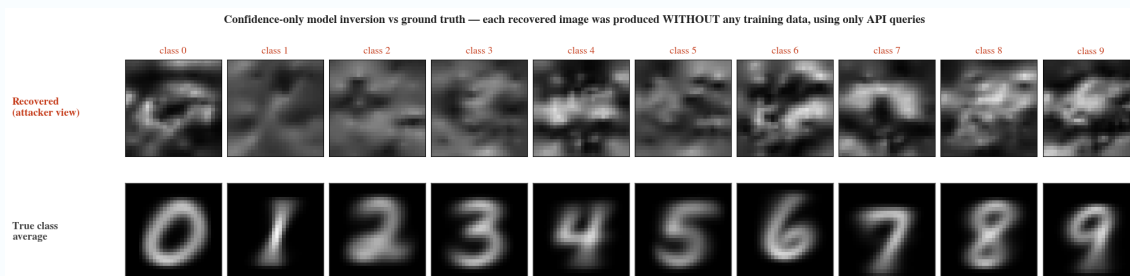
```

Inversion status : all 10 digit classes reconstructed
Steps per class  : 1,000
Trajectory class  : class 0
Snapshot steps    : 0, 50, 100, 200, 500, 1000

```



Class-0 inversion trajectory. The image starts from noise and quickly becomes a high-confidence class-0 prototype.



Recovered digit prototypes compared with true class averages. The top row shows the attacker's recovered images; the bottom row shows the true average MNIST digit for each class.

The trajectory shows that class-0 confidence rises from 0.008 at step 0 to 1.000 after 50 steps, even though the image is still visually imperfect. This highlights an important point: model confidence and human visual quality are not the same.

Across all 10 digits, the recovered images are best understood as model-specific class prototypes rather than exact training examples. They still leak information because they reveal which visual patterns strongly activate each class.

Listing 7: Cell 7: Measure reconstruction quality

```

1 for digit in range(10):
2     # True class prototype: average MNIST training image for this digit.
3     ...
4
5 mean_mse = float(np.mean(mse_per_class))

```

MSE and confidence summary

Example Output

Class	MSE	Max confidence at recovery
0	0.058944	100.0%
1	0.079869	100.0%
2	0.069664	100.0%
3	0.073574	100.0%
4	0.080116	100.0%
5	0.084994	100.0%
6	0.097152	100.0%
7	0.065632	100.0%
8	0.076620	100.0%
9	0.074712	100.0%

Mean MSE across all classes : 0.076128

Lower MSE : closer to the true class prototype

Every recovered image reaches nearly 100% confidence for its target class, so the optimization successfully finds high-confidence inputs. However, confidence and visual quality are not the same. A noisy image can still receive 100% model confidence if it activates the features the model associates with that class.

The mean MSE is 0.076128 in this run. Lower MSE means the recovered image is closer to the true class average, but MSE should be interpreted together with the visual comparison above.

Exercise 2: Image Inversion Analysis

1. Look at the trajectory for class 0. What changes between step 0 and step 50? What changes more slowly between step 50 and step 1000?
2. Record your mean MSE: _____. Which digit has the lowest MSE? Which digit has the highest MSE?
3. Compare the recovered images with the true class averages. Which recovered digits are visually easiest to recognize? Which are the most ambiguous?
4. The recovered images reach nearly 100% confidence, but some still look noisy. What does this tell you about the difference between model confidence and human visual quality?
5. The recovered images do not look like one specific person's handwriting. They look more like class-level patterns. Why does this happen when the attack optimizes for a digit class rather than for a particular individual?
6. **Scene A: Fewer gradient steps.** Rerun the inversion for digit 0 with `n_steps=100`. Is the digit recognizable? How does the MSE compare with the 1,000-step result?
7. **Scene B: Different initialization.** Try initializing `x_inv` with a uniform gray image instead of Gaussian noise. Does it converge to a similar pattern? Why might different initializations lead to different high-confidence images?

8. **Privacy reflection:** MNIST digits are not sensitive personal records. In a face recognition, medical imaging, or genetic-risk model, what would make the same type of inversion attack more serious?

4 Part 3: Why Confidence Scores Are the Attacker's Fuel

4.1 The label-only baseline

Before applying defenses, we first isolate why confidence scores are so useful to the attacker.

If an API returns the full confidence vector, the attacker can see whether a small input change increases or decreases $P(\text{class } c)$. That creates a useful gradient-like signal. If the API returns only the predicted label, the attacker sees only whether the current image is classified as class c or not. The search becomes much closer to a random walk.

Listing 8: Cell 8: Label-only inversion baseline

```

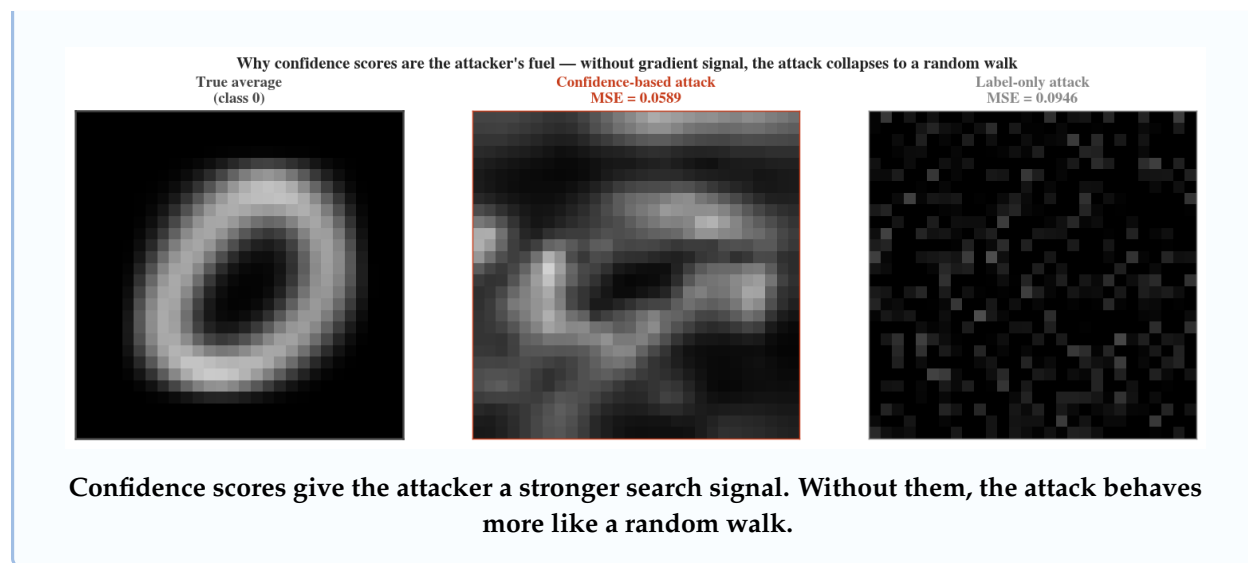
1 def invert_label_only(model, target_class, n_steps=1000,
2                       img_shape=(1, 1, 28, 28)):
3     """
4     Random-walk inversion.
5
6     The attacker observes only the predicted label, not the
7     confidence score. Without confidence values, there is no
8     smooth direction for gradient ascent.
9     """
10    model.eval()
11    ...

```

What the output should look like

Example Output

Target digit	:	class 0
Confidence-based MSE	:	0.058944
Label-only MSE	:	0.094635
Label-only / confidence-based ratio	:	1.61×



The confidence-based attack produces a clearer class-0 pattern and a lower MSE. The label-only attack has no confidence signal to climb, so it mostly accepts random moves that happen to preserve the target label.

Exercise 3: The Value of Confidence Scores

1. Record your MSE values:
 Confidence-based inversion MSE: _____
 Label-only inversion MSE: _____
 Ratio, label-only/confidence: _____ ×
2. Compare the two recovered images. Which one looks more like the class-0 average?
3. Why does the confidence-based attack have a more useful search direction than the label-only attack?
4. The label-only attack still sometimes finds images classified as class 0. Why does that not necessarily mean it has recovered a good class prototype?
5. **Key question:** What is the simplest API-level defense suggested by this experiment?

5 Part 4: Defense 1 — Confidence Score Suppression

5.1 The idea

A simple defense is to restrict what the API returns. Instead of exposing the full 10-dimensional probability vector, the API can round scores or return only the top- k entries.

This defense matters most in a black-box setting. A white-box attacker can compute exact gradients from the model weights, so output suppression does not block the optimization. A black-box attacker, however, must estimate gradients from API outputs. If those outputs are rounded or truncated, the gradient estimate becomes noisy, coarse, or completely uninformative.

In this section, we compare two attackers:

- **White-box upper bound:** the attacker has full gradient access. This shows what suppression cannot stop.
- **Black-box NES attacker:** the attacker only observes the suppressed API output and estimates gradients from queries.

We test four API modes:

- **No defense:** return the full probability vector.
- **Round to 2 decimals:** return all scores, rounded to two decimal places.
- **Round to 1 decimal:** return all scores, rounded to one decimal place.
- **Top-3 only:** return only the three largest class scores.

The key tradeoff

Confidence suppression can reduce privacy leakage, but it also reduces utility. A user may need probability scores to understand uncertainty, compare alternatives, or make risk-sensitive decisions. Returning less information makes attacks harder, but it also makes the API less informative.

Listing 9: Cell 9: Confidence suppression and inversion attackers

```

1 def apply_suppression(probs, mode):
2     """
3     Simulate what the API returns.
4
5     none      : full probability vector
6     round2    : probabilities rounded to 2 decimals
7     round1    : probabilities rounded to 1 decimal
8     top3      : keep only the top-3 probabilities
9     """
10    ...
11
12
13 def invert_whitebox(model, target_class, n_steps=1000, lr=0.1,
14                    tv_lambda=0.05, img_shape=(1, 1, 28, 28)):
15     """
16     White-box upper bound.
17
18     The attacker has model gradients, so API suppression does not
19     affect the reconstruction.
20     """
21     ...
22
23
24 def invert_blackbox(model, target_class, suppression='none',
25                    n_steps=600, lr=0.5, eps=0.05,
26                    n_samples=40, img_shape=(1, 1, 28, 28)):
27     """
28     Black-box NES inversion.
29
30     The attacker estimates the gradient by querying the API at
31      $x + \text{eps} \cdot u$  and  $x - \text{eps} \cdot u$ . The only signal available is the
32     suppressed API output.
33     """
34     ...

```


Listing 10: Cell 10: Compare suppression modes

```

1 target_class = 0
2 modes = ['none', 'round2', 'round1', 'top3']
3 ...
4
5 # Random-initialization baseline.
6 # If black-box MSE is near this value, the attack is frozen.
7 torch.manual_seed(42)
8 ...

```

What the output should look like

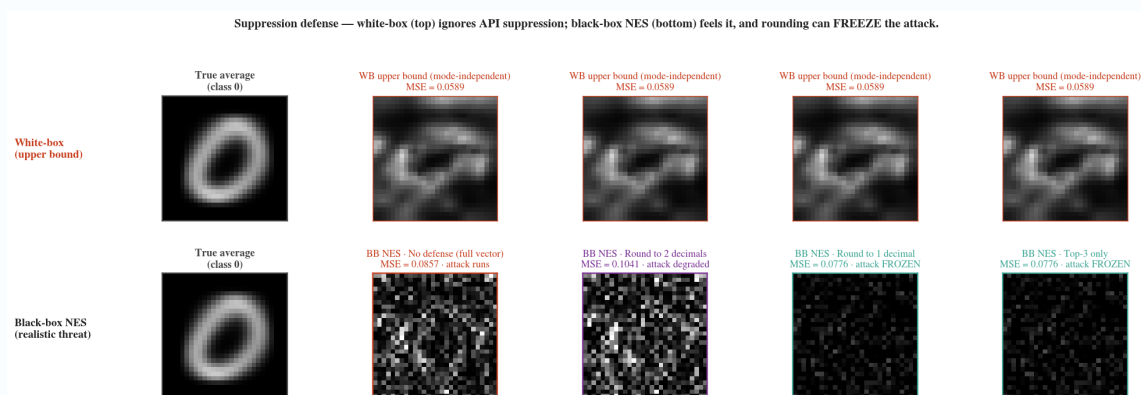
Example Output

API mode	White-box MSE	Black-box MSE	BB / WB
No defense	0.0589	0.0857	1.45×
Round to 2 decimals	0.0589	0.1041	1.77×
Round to 1 decimal	0.0589	0.0776	1.32×
Top-3 only	0.0589	0.0776	1.32×

```

Frozen-attack      : MSE = 0.0776
baseline
No defense         : black-box attack runs
Round to 2         : attack degraded
decimals
Round to 1 decimal : attack frozen near
                    initialization
Top-3 only         : attack frozen near
                    initialization

```



Suppression affects black-box attackers much more than white-box attackers. Rounding or truncating the API output can remove the signal needed for NES gradient estimation.

The white-box row is identical across modes because a white-box attacker does not depend on the API output. It is an upper bound showing what output suppression cannot prevent.

The black-box NES row changes across modes because the attacker estimates gradients from returned confidence scores. Rounding to 1 decimal and top-3 output freeze the attack near its random initialization in this run.

Exercise 4: Confidence Suppression Results

Fill in the comparison table:

API mode	White-box MSE	Black-box MSE	Attack status
No defense	_____	_____	_____
Round to 2 decimals	_____	_____	_____
Round to 1 decimal	_____	_____	_____
Top-3 only	_____	_____	_____

1. Why are the white-box MSE values the same across all API modes?
2. Which suppression mode gives the largest black-box MSE? What does that mean visually?
3. The frozen baseline is 0.0776 in this run. Which black-box attacks are close to that baseline?
4. Why can rounding to 1 decimal freeze the NES attack even though the model itself is unchanged?
5. What is the practical lesson for deploying a model API: should it return full confidence scores, rounded scores, top- k scores, or labels only? Explain the privacy-utility tradeoff.

6 Part 5: Defense 2 — Output Perturbation

6.1 The idea

Another defense is to add random noise to the confidence scores before returning them. Instead of hiding entries or rounding them, the API returns a perturbed probability vector.

Output perturbation:

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$$

The attacker observes $\tilde{f}(\mathbf{x})$, not the clean confidence vector $f(\mathbf{x})$.

A larger σ gives stronger protection because it makes black-box gradient estimation noisier. But larger noise can also damage legitimate API utility, so the goal is to find a useful privacy-utility balance.

Listing 11: Cell 11: Black-box NES under output perturbation

```

1 def invert_with_noise(model, target_class, sigma=0.05,
2                       n_steps=600, lr=0.5, eps=0.05,
3                       n_samples=40, img_shape=(1, 1, 28, 28)):
4     """

```

```

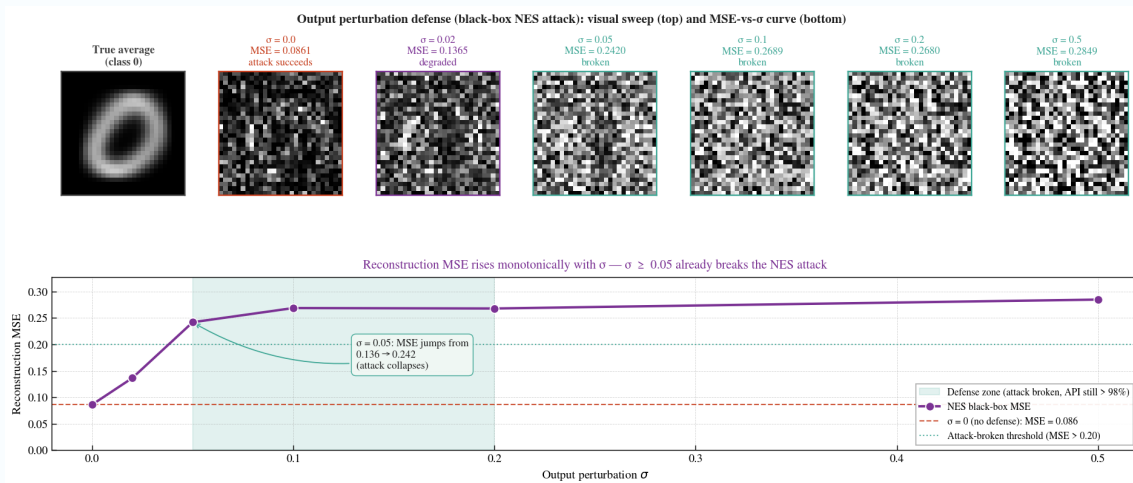
5   Black-box NES inversion under Gaussian output perturbation.
6
7   The API adds  $N(0, \sigma^2)$  noise to the target confidence
8   before returning it. The attacker estimates gradients only
9   from these noisy API responses.
10  """
11  model.eval()
12  ...

```

What the output should look like

Example Output

Output noise σ	Reconstruction MSE	Interpretation
0.00	0.086110	attack succeeds
0.02	0.136469	visibly degraded
0.05	0.242009	attack broken
0.10	0.268877	attack broken
0.20	0.268035	attack broken
0.50	0.284899	attack broken



Output perturbation breaks the black-box NES attack once the noise is large enough. In this run, $\sigma = 0.05$ already pushes the reconstruction MSE above the attack-broken threshold.

The no-defense case has MSE 0.0861, and the recovered image still contains a class-0 pattern. As σ increases, the NES gradient estimate becomes less reliable, and the reconstruction quickly turns into noise.

At $\sigma = 0.05$, the MSE jumps to 0.2420, which is above the attack-broken threshold used in the figure. This suggests that modest output noise can strongly disrupt this black-box inversion attack.

Listing 12: Cell 12: Measure API utility under output noise

```

1  def api_quality_under_noise(model, loader, sigma):
2      """

```

```

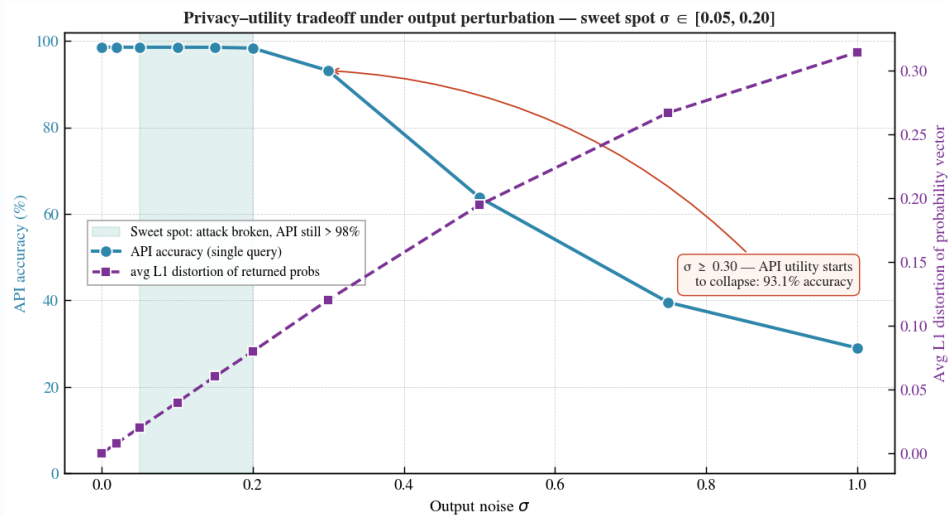
3     Measure legitimate API quality under one noisy query.
4
5     Returns:
6         accuracy          : argmax accuracy after output noise
7         avg_l1_dist       : average L1 distortion of the probability vector
8         """
9         ...
10
11     with torch.no_grad():
12         ...
13     ...

```

What the output should look like

Example Output

σ	API accuracy	Accuracy drop	Avg. L1 distortion
0.00	98.55%	0.00 pp	0.0000
0.02	98.56%	-0.01 pp	0.0081
0.05	98.54%	0.01 pp	0.0201
0.10	98.54%	0.01 pp	0.0399
0.15	98.53%	0.02 pp	0.0603
0.20	98.34%	0.21 pp	0.0799
0.30	93.13%	5.42 pp	0.1204
0.50	63.82%	34.73 pp	0.1946
0.75	39.56%	58.99 pp	0.2670
1.00	29.02%	69.53 pp	0.3144



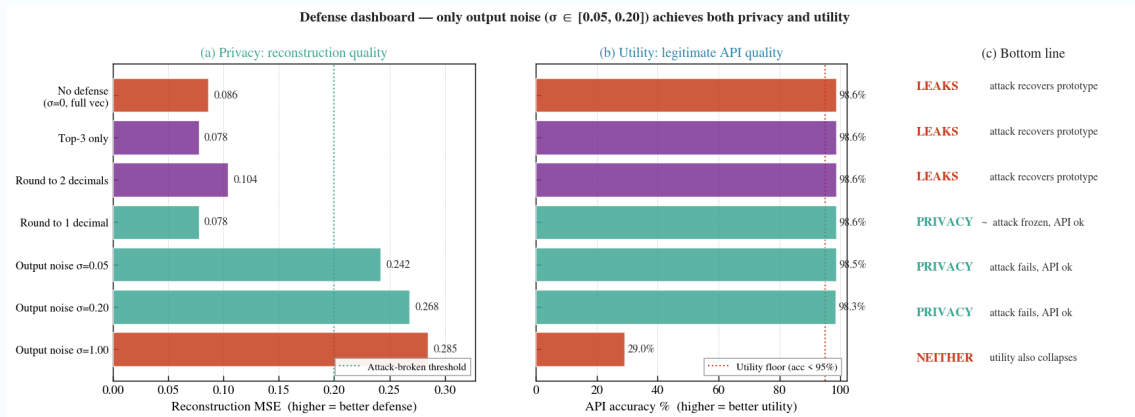
Privacy–utility tradeoff under output perturbation. Moderate noise can break the attack while preserving API accuracy, but large noise collapses utility.

The useful range is $\sigma \in [0.05, 0.20]$: the black-box attack is already broken, while API accuracy remains above 98%. When σ reaches 0.30 or higher, the API accuracy starts to drop sharply, so the

defense becomes too costly.

Putting the defenses side by side

Example Output



Defense dashboard: the best defense is not the one with the highest MSE, but the one that improves privacy while keeping API utility high.

The dashboard shows why output noise around $\sigma = 0.05$ to 0.20 is a useful operating region in this experiment. It increases reconstruction MSE enough to break the attack while keeping API accuracy near the no-defense level.

Very large noise, such as $\sigma = 1.00$, gives high reconstruction MSE but destroys API accuracy, so it is not a practical defense.

Exercise 5: Output Perturbation Results

- Fill in the tradeoff table:

σ	Inversion MSE	API accuracy	Acc. drop
0.00	_____	_____ %	—
0.02	_____	_____ %	_____ pp
0.05	_____	_____ %	_____ pp
0.10	_____	_____ %	_____ pp
0.20	_____	_____ %	_____ pp
0.30	_____	_____ %	_____ pp

- At what value of σ does the attack first become broken according to the MSE threshold?
- Which range of σ gives the best privacy–utility balance in this experiment? Explain using both reconstruction MSE and API accuracy.
- Why is $\sigma = 1.00$ not a good defense, even though the reconstruction MSE is high?

5. An adaptive attacker who knows σ can average repeated noisy queries to reduce the noise. How many independent queries are needed to reduce the noise standard deviation by a factor of 10? Hint: standard deviation decreases as $1/\sqrt{n}$.
6. Compare confidence suppression and output perturbation. Which one is more effective against the black-box NES attack in this experiment? Which one is easier to deploy? Which one has the clearer utility cost?

7 Putting It All Together

7.1 The complete defense comparison

API setting	Inv. MSE	API acc.	Formal?	Main takeaway
Full confidence, no defense	0.086	98.55%	No	Attack succeeds; full confidence gives the attacker the strongest signal.
Round to 2 decimals	0.104	98.55%	No	Slightly degrades the black-box attack, but does not reliably stop it.
Top-3 only	0.078	98.55%	No	Reduces output information, but protection depends on the attacker and query strategy.
Round to 1 decimal	0.078	98.55%	No	The NES attack freezes near initialization in this run, but this is still an empirical defense.
Output noise $\sigma = 0.05$	0.242	98.54%	No	Breaks the NES attack while preserving normal API accuracy.
Output noise $\sigma = 0.20$	0.268	98.34%	No	Still breaks the attack with very small utility loss.
Output noise $\sigma = 1.00$	—	29.02%	No	Not practical; utility collapses even if reconstruction becomes poor.

7.2 A practical decision guide

Setting	Recommended approach
Public API, low sensitivity	Return only the confidence information needed for the task. Rounding to 2 decimals can be a cheap first step, but it should not be treated as a strong privacy defense.
Medical / financial data	Avoid exposing full confidence vectors. Use label-only or top- k outputs when possible, and consider moderate output perturbation if confidence scores must be returned.
Formal privacy required	Combine output-side controls with DP-SGD training from Membership Inference Attacks . DP-SGD limits individual-record influence during training, while output perturbation weakens the API signal used for inversion.
Internal trusted users	API, Monitor query volume, rate-limit repeated probing, and audit suspicious query patterns. Output defenses help, but access control and monitoring are also necessary.

Exercise 6: Final Reflection

1. Return to your prediction from the opening Think box. What did you expect gradient ascent to recover? How accurate was your prediction? What surprised you most?
2. **Complete the five-lab series map:**

Online lab	Attack	What is stolen or damaged	Key defense
Adversarial Examples	FGSM / PGD	Model integrity at inference time	_____
Data Poisoning Attacks	Poisoned training data	Model behavior through manipulated training data	_____
Backdoor Attacks	Patch / blend / invisible triggers	Hidden trigger-based model behavior	_____
Membership Inference Attacks	Confidence-based membership attack	Whether a record was in the training set	_____
Model Inversion Attacks	Confidence-based reconstruction	Private input features or class prototypes	_____

3. The defenses in this lab are all *output-side*: they restrict or perturb what the API returns. **Membership Inference Attacks** introduced a *training-side* defense, DP-SGD. Can DP-SGD also reduce model inversion risk? How would it help?
4. **The fundamental tension:** A model with sharper, more confident class regions can sometimes be easier to invert, because the confidence landscape gives the attacker a clearer direction. Does this mean we should intentionally make models less accurate to protect privacy? Or should we control what the API exposes?
5. **Ethics:** The warfarin dosage model in the Fredrikson 2014 paper was a real clinical deployment. The researchers demonstrated that it was invertible and notified the model owners. Should security researchers be allowed to test production medical AI systems for vulnerabilities without prior authorization? What framework—legal, ethical, or institutional—would you propose?

8 Summary

Concept	What it does	Key insight
Model inversion attack	Reconstructs private features or class prototypes from model outputs	Confidence scores can act as a compass toward sensitive information
Gradient ascent	Iteratively maximizes target-class confidence	Moves toward a class prototype rather than across a decision boundary
Reconstruction quality	Measures distance between recovered input and true class prototype	Lower MSE means more private information was recovered
Confidence suppression	Restricts, rounds, or truncates confidence outputs	Most useful against black-box attackers that estimate gradients from API responses
Output perturbation	Adds Gaussian noise to confidence scores before returning them	Can break black-box inversion, but too much noise harms API utility
Privacy-utility tradeoff	Balances attack resistance against normal API usefulness	A defense is practical only if it improves privacy without destroying utility

Primary references

Fredrikson, M., Lantz, E., Jha, S., Lin, S., Page, D., & Ristenpart, T. (2014). Privacy in pharmacogenetics: An end-to-end case study of personalized warfarin dosing. *USENIX Security Symposium 2014*. [usenix.org](https://www.usenix.org/conference/usenix-security-symposium-2014)

Fredrikson, M., Jha, S., & Ristenpart, T. (2015). Model inversion attacks that exploit confidence information and basic countermeasures. *ACM CCS 2015*, pages 1322–1333. [doi:10.1145/2810103.2813677](https://doi.org/10.1145/2810103.2813677)

Further reading

Zhang, C., et al. (2020). The secret revealer: Generative model-inversion attacks against deep neural networks. *CVPR 2020*. [arXiv:1911.07135](https://arxiv.org/abs/1911.07135) (GMI: first GAN-based model inversion; shows stronger models are more vulnerable)

Kahla, M., et al. (2022). Label-only model inversion attacks via boundary repulsion. *CVPR 2022*. [arXiv:2203.01925](https://arxiv.org/abs/2203.01925) (BREP-MI: inversion from labels only, no confidence needed)

Nguyen, T.-T., et al. (2024). Re-thinking model inversion attacks: the role of data augmentation. *AAAI 2024*. [arXiv:2304.01669](https://arxiv.org/abs/2304.01669) (modern analysis of model inversion attacks and defenses)